Overview of Graph attention network

Subject: Graph attention network

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

1.

Scalability and distributed training Training GNNs on large graphs presents computational challenges that differ substantially from those in other neural network architectures. Two broad problem settings arise: training on a single very large graph (such as a social network with billions of nodes), and training on collections of large individual graphs (such as molecular systems requiring higher-order interaction modeling). For the single large graph setting, sampling-based methods reduce memory requirements by operating on subgraphs rather than the full graph. Cluster-GCN partitions the graph into subgraphs using graph partitioning algorithms, training the model on each partition in sequence. GraphSAINT instead samples subgraphs stochastically during training, constructing unbiased estimators for the full graph loss. Distributed frameworks such as DistDGL extend training to multi-machine settings, managing the inter-machine communication required for neighborhood aggregation when a graph's nodes are distributed across machines. For tasks such as atomic simulations, a different bottleneck arises: GNN architectures that model higher-order interactions between triplets or quadruplets of atoms are memory-intensive at the level of individual graphs, making standard data parallelism — where each GPU processes a separate sample — insufficient when individual graphs exceed single-device memory. Graph Parallelism addresses this by distributing a single input graph across multiple GPUs, partitioning nodes and edges across devices rather than distributing samples across devices.. This approach enabled GNNs with hundreds of millions to billions of parameters to be trained on atomic systems that would otherwise exceed single-device memory, and has been applied to the development of large-scale universal interatomic potentials.

2.

Graph attention network The graph attention network (GAT) was introduced by Petar Veli■kovi■ et al. in 2018. A graph attention network is a combination of a GNN and an attention layer. The implementation of attention layer in graphical neural networks helps provide attention or focus to the important information from the data instead of focusing on the whole data. A multi-head GAT layer can be expressed as follows:

3.

α u v = exp ■ (LeakyReLU (a T [W x u ■ W x v ■ e u v])) ∑ z ∈ N u exp ■ (LeakyReLU (a T [W x u ■ W x z ■ e u z])) {\displaystyle \alpha _{uv}={\frac {\exp({\text{LeakyReLU}}\left(\mathbf {a} ^{T}[\mathbf {W} \mathbf {x} _{u}\Vert \mathbf {W} \mathbf {x} _{v}\Vert \mathbf {e} _{uv}]\right))}{\sum _{z\in N_{u}}\exp({\text{LeakyReLU}}\left(\mathbf {a} ^{T}[\mathbf {W} \mathbf {x} _{u}\Vert \mathbf {W} \mathbf {x} _{z}\Vert \mathbf {e} _{uz}]\right))}}}

4.

To represent an image as a graph structure, the image is first divided into multiple patches, each of which is treated as a node in the graph. Edges are then formed by connecting each node to its nearest neighbors based on spatial or feature similarity. This graph-based representation enables the application of graph learning models to visual tasks. The relational structure helps to enhance feature extraction and improve performance on image understanding.

5.

where a {\displaystyle \mathbf {a} } is a vector of learnable weights, · T {\displaystyle \cdot ^{T}} indicates transposition, e u v {\displaystyle \mathbf {e} _{uv}} are the edge features (if present), and LeakyReLU {\displaystyle {\text{LeakyReLU}}} is a modified ReLU activation function. Attention coefficients are then normalized via softmax to make them easily comparable across different nodes. A GCN can be seen as a special case of a GAT where attention coefficients are not learnable, but fixed and equal to the edge weights w u v {\displaystyle w_{uv}} .

6.

Heterophilic Graph Learning Homophily principle, i.e., nodes with the same labels or similar attributes are more likely to be connected, has been commonly believed to be the main reason for the superiority of Graph Neural Networks (GNNs) over traditional Neural Networks (NNs) on graph-structured data, especially on node-level tasks. However, recent work has identified a non-trivial set of datasets where GNN's performance compared to the NN's is not satisfactory. Heterophily, i.e., low homophily, has been considered the main cause of this empirical observation. People have begun to revisit and re-evaluate most existing graph models in the heterophily scenario across various kinds of graphs, e.g., heterogeneous graphs, temporal graphs and hypergraphs. Moreover, numerous graph-related applications are found to be closely related to the heterophily problem, e.g. graph fraud/anomaly detection, graph adversarial attacks and robustness, privacy, federated learning and point cloud segmentation, graph clustering, recommender systems, generative models, link prediction, graph classification and coloring, etc. In the past few years, considerable effort has been devoted to studying and addressing the heterophily issue in graph learning.

7.

Graph neural networks are one of the main building blocks of AlphaFold, an artificial intelligence program developed by Google's DeepMind for solving the protein folding problem in biology. AlphaFold achieved first place in several CASP competitions.

8.

Message passing layers are permutation-equivariant layers mapping a graph into an updated representation of the same graph. Formally, they can be expressed as message passing neural networks (MPNNs). Let G = (V , E) {\displaystyle G=(V,E)} be a graph, where V {\displaystyle V} is the node set and E {\displaystyle E} is the edge set. Let N u {\displaystyle N_{u}} be the neighbourhood of some node u ∈ V {\displaystyle u\in V} . Additionally, let x u {\displaystyle \mathbf {x} _{u}} be the features of node u ∈ V {\displaystyle u\in V} , and e u v {\displaystyle \mathbf {e} _{uv}} be the features of edge (u , v) ∈ E {\displaystyle (u,v)\in E} . An MPNN layer can be expressed as follows:

9.

GNNs are used as fundamental building blocks for several combinatorial optimization algorithms. Examples include computing shortest paths or Eulerian circuits for a given graph, deriving chip placements superior or competitive to handcrafted human solutions, and improving expert-designed branching rules in branch and bound.

10.

h u = ■ k = 1 K σ (∑ v ∈ N u α u v k W k x v) {\displaystyle \mathbf {h} _{u}={\overset {K}{\underset {k=1}{\Big \Vert }}}\sigma \left(\sum _{v\in N_{u}}\alpha _{uv}^{k}\mathbf {W} ^{k}\mathbf {x} _{v}\right)}

11.

where K {\displaystyle K} is the number of attention heads, ■ {\displaystyle {\Big \Vert }} denotes vector concatenation, σ (·) {\displaystyle \sigma (\cdot)} is an activation function (e.g., ReLU), N u {\displaystyle N_{u}} is the set of immediate neighbor nodes of node u {\displaystyle u} , including node u {\displaystyle u} itself, α u v k {\displaystyle \alpha _{uv}^{k}} are attention coefficients for the k {\displaystyle k} -th attention head, and W k {\displaystyle W^{k}} is a matrix of trainable parameters for the k {\displaystyle k} -th attention head. For the final GAT layer, the outputs from each attention head are averaged before the application of the activation function. Formally, the final GAT layer can be written as: